1. Problem Title & Source

Title: **Sort Items Based on Risk Severity**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Short Summary)

At an airport security checkpoint, several passenger items are confiscated and placed in a large box.

Each item has a **risk severity level** represented as:

0 → Low risk

1 → Medium risk

2 → High risk

You are given an array of N integers representing the risk level of items.

Your task is to **sort the array so that all items are arranged in increasing order of risk severity**.

Final order:

0 → 1 → 2

3. Examples

Example 1

Input

7

1 0 2 0 1 0 2

Output

0 0 0 1 1 2 2

Explanation

Items are arranged according to risk severity:

Low risk → Medium risk → High risk

Example 2

Input

10

2 1 0 2 1 0 0 1 2 0

Output

0 0 0 0 1 1 1 2 2 2

4. Constraints

$1 \le N \le 100$

$arr[i] \in \{0,1,2\}$

Observation

Only **three possible values** exist.

5. Core Concept (Pattern)

Dutch National Flag Algorithm

Three-way partitioning.

This algorithm was introduced by **Edsger Dijkstra**.

It solves problems where an array contains **three distinct categories**.

6. Thought Process

Brute Force Idea

Sort the array using a standard sorting algorithm.

Example

Arrays.sort()

Time complexity

O(n log n)

But this problem can be solved faster.

Better Idea

Count occurrences.

Steps:

count0

count1

count2

Then reconstruct the array.

Time complexity

O(n)

But it requires **two passes**.

Optimal Idea (Dutch National Flag)

Use **three pointers**:

low

mid

high

Meaning

0 → before low

1 → between low and mid

unknown → mid to high

2 → after high

7. Visualization

Example

[1,0,2,0,1,0,2]

Pointers

low = 0

mid = 0

high = 6

Steps

1 → mid++

0 → swap(low,mid)

2 → swap(mid,high)

Final array

0 0 0 1 1 2 2

8. Pseudocode

```
low = 0
mid = 0
high = n-1

while mid <= high

   if arr[mid] == 0
      swap(arr[low], arr[mid])
      low++
      mid++

   else if arr[mid] == 1
      mid++

   else
      swap(arr[mid], arr[high])
      high--
```

9. Code Implementation

Python

```
n = int(input())
arr = list(map(int,input().split()))

low = 0
mid = 0
high = n - 1

while mid <= high:

    if arr[mid] == 0:
        arr[low], arr[mid] = arr[mid], arr[low]
        low += 1
        mid += 1

    elif arr[mid] == 1:
        mid += 1

    else:
        arr[mid], arr[high] = arr[high], arr[mid]
        high -= 1

print(*arr)
```

Java

```
import java.util.*;

class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n];

        for(int i=0;i<n;i++)
```

```
            arr[i] = sc.nextInt();

        int low = 0;
        int mid = 0;
        int high = n-1;

        while(mid <= high){

            if(arr[mid] == 0){

                int temp = arr[low];
                arr[low] = arr[mid];
                arr[mid] = temp;

                low++;
                mid++;
            }

            else if(arr[mid] == 1){
                mid++;
            }

            else{

                int temp = arr[mid];
                arr[mid] = arr[high];
                arr[high] = temp;

                high--;
            }
        }

        for(int x : arr)
            System.out.print(x + " ");
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n)

Single pass through the array.

Space Complexity

O(1)

In-place sorting.

11. Common Mistakes

1. Incrementing mid after swapping with high
2. Incorrect pointer boundaries
3. Using extra array unnecessarily
4. Confusing pointer roles

Correct rule

When swapping with high → do NOT increment mid

12. Detailed Dry Run

Input

1 0 2 0 1 0 2

Step	Array	low	mid	high
1	1 0 2 0 1 0 2	0	1	6
2	0 1 2 0 1 0 2	1	2	6
3	0 1 2 0 1 0 2	1	2	5
4	0 0 1 0 1 2 2	2	3	5
5	0 0 0 1 1 2 2	3	4	5

Final result

0 0 0 1 1 2 2

13. Real-Life Applications

Three-way partitioning is used in:

- Data classification
- Image segmentation
- Color sorting

- Traffic control simulations

14. Common Traps

1. Using Arrays.sort() unnecessarily
2. Incrementing mid incorrectly
3. Missing mid <= high condition
4. Incorrect swap operations

15. Related Problems

Important interview problems using the same pattern:

LC 75 Sort Colors

LC 283 Move Zeroes

LC 27 Remove Element

16. Alternate Approaches

Approach	Complexity	Notes
Sorting	O(n log n)	unnecessary
Counting	O(n)	two passes
Dutch National Flag	O(n)	optimal

17. Why This Solution Works

The algorithm maintains **three partitions** in the array:

0s region

1s region

2s region

By dynamically expanding these regions using pointers, we guarantee that each element is processed **exactly once**.

18. Variations / Follow-Up Questions

Possible interview follow-ups:

1. Sort an array containing **0,1,2,3**
2. Partition array around a pivot
3. Separate **even and odd numbers**
4. Three-way quicksort partition